

Python Programming Report 6

Name: Amira Mahmoud Mostafa Al

1. Python Encapsulation

Encapsulation is a core OOP concept that restricts direct access to an object's data. It bundles variables and methods together into a single unit and hides the internal state to prevent accidental changes.

1.1 Private Properties

In Python, you can make a property "private" by adding a double underscore `__` before its name. This prevents anyone from accessing or changing it directly from outside the class.

Example:

```
1 class ExamResult:
2     def __init__(self, student_name, score):
3         self.student_name = student_name
4         self.__score = score # Private property
5
6 result = ExamResult("Amira Mahmoud", 98)
7 print(result.student_name)
8
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, lexcludeText, t...

[Running] python -u "a:\python\Task_6.py"

Amira Mahmoud

1.2 Getters and Setters

To safely interact with private data, we use:

- **Getter:** A method to read the private value.
- **Setter:** A method to modify the private value, allowing us to add **validation logic** (like checking if a score is between 0 and 100) before saving it.

Example:

```
11 class ExamResult:
12     def __init__(self, score):
13         self.__score = score
14
15     def get_score(self): # Getter
16         return self.__score
17
18     def set_score(self, value):
19         if 0 <= value <= 100:
20             self.__score = value
21         else:
22             print("Error: Invalid score!")
23
24 res = ExamResult(95)
25 res.set_score(105) #invalid score
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Filter (e.g. text, !excludeTe

```
[Running] python -u "a:\python\Task_6.py"
Error: Invalid score!
```

2. Abstraction in OOP

Abstraction hides complex implementation details and only shows the necessary features to the user. It focuses on **what** an object does rather than **how** it does it.

2.1 Abstract Base Classes (ABC)

An abstract class acts as a blueprint for other classes. You cannot create an object from it directly; instead, it forces child classes to implement specific methods using the `@abstractmethod` decorator.

Why use it?

- **Consistency:** Ensures all subclasses have the same required methods.
- **Security:** Hides internal complexity.
- **Simplicity:** Reduces overall code complexity.

Example (Payment Methods):

```
27
28 from abc import ABC, abstractmethod
29
30 class Payment(ABC):
31     @abstractmethod
32     def process(self, amount):
33         pass
34
35 class VisaCard(Payment):
36     def process(self, amount):
37         print(f"Processing Visa payment of ${amount}")
38
39 class PayPal(Payment):
40     def process(self, amount):
41         print(f"Processing PayPal payment of ${amount}")
42
43 pay1 = VisaCard()
44 pay1.process(100)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Filter (e.g. text, lexcludeText, t...

Code

[Running] python -u "a:\python\Task_6.py"

Processing Visa payment of \$100